Two-Stage Retrieval-Augmented Generation: Mitigating Dense Embedding Failure Modes via Cross-Encoder Re-Ranking

Comprehensive Technical Report & Failure Mode Analysis

Student Author: Raj K. Jain | **Course:** Large Language Models & AI Systems (LMA)
Deliverables: GitHub Repository + Technical Report | **Date:** October 2026

Abstract

Retrieval-Augmented Generation (RAG) grounds Large Language Models (LLMs) in verifiable private corpora. However, single-stage dense retrieval using Bi-Encoders suffers from an informational bottleneck: compressing text passages into fixed-dimensional vectors obscures syntactic modifiers, conditional restrictions, and negative constraints. In this study, we contrast a single-stage dense retrieval baseline (`all-MiniLM-L6-v2`) against a two-stage re-ranking architecture deploying a Cross-Encoder (`ms-marco-MiniLM-L-6-v2`) over a complex 4-page enterprise travel and expense policy document (30 clauses). We demonstrate that the Cross-Encoder successfully promotes critical policy exceptions and constraints buried at Rank #4 by the Bi-Encoder directly into the Top-2, preventing severe LLM hallucinations and policy misstatements. Empirical latency profiling across 10 warmed iterations demonstrates an average re-ranking overhead of 18.05 ms, yielding a total pipeline latency of 25.58 ms—well within real-time interactive thresholds (< 100 ms). Finally, we provide a formal computational scaling analysis explaining why Cross-Encoders cannot be deployed across the full corpus from scratch.

1 Architecture Comparison: Bi-Encoder vs. Cross-Encoder

The central challenge in Information Retrieval (IR) for RAG lies in estimating the relevance $S(q, c)$ between an incoming query q and a candidate passage c. Bi-Encoders and Cross-Encoders solve this problem through fundamentally distinct neural mechanisms.

Architectural Formulations: Bi-Encoder vs. Cross-Encoder

Bi-Encoder (Independent Encoding / Dense Vectors):

$$\vec{u} = \text{Pool}(\text{Transformer}_\theta(q)) \in \mathbb{R}^d, \quad \vec{v} = \text{Pool}(\text{Transformer}_\theta(c)) \in \mathbb{R}^d$$

$$S_{\text{Bi}}(q, c) = \cos(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\|_2 \, \|\vec{v}\|_2}$$

Key Property: Query tokens and chunk tokens **never interact**. Representations are fully decoupled.

Cross-Encoder (Joint All-to-All Multi-Head Self-Attention):

$$X = [\text{CLS}] \circ q \circ [\text{SEP}] \circ c \circ [\text{SEP}]$$

$$H = \text{TransformerLayers}(X) \in \mathbb{R}^{L \times d}$$

$$S_{\text{Cross}}(q, c) = \mathbf{W}^T h_{\text{CLS}} + b \in (-\infty, +\infty)$$

Key Property: Every query token attends directly to every chunk token across all self-attention layers.

1.1 Mechanical Differences

1. **Token-Level Cross-Attention vs. Embedding Bottleneck:** In a Bi-Encoder, the document passage c is compressed into a single vector $\vec{v} \in \mathbb{R}^{384}$. Squeezing an entire 300-word paragraph containing general rules, conditional waivers, and monetary penalties into 384 numbers creates an extreme semantic bottleneck. At inference time, the query vector $\vec{u}$ only measures coarse geometric alignment with $\vec{v}$. In contrast, a Cross-Encoder concatenates q and c, allowing bidirectional self-attention $\text{softmax}(QK^T/\sqrt{d_k})V$ to directly correlate specific query terms (e.g., *"client negotiations"*) with subtle conditional clauses inside the passage (e.g., *"within 12 hours of flight arrival"*).

2. **Indexability and Precomputation:** Because the Bi-Encoder produces document vectors $\vec{v}_i$ independently of any query, all N chunks in a corporate corpus can be computed offline once and indexed into an Approximate Nearest Neighbor (ANN) index (e.g., HNSW or IVF in FAISS). Query matching requires

only a single vector dot product search scaling in sub-linear time $\mathcal{O}(\log N)$. For a Cross-Encoder, however, the joint input $[\text{CLS}] \circ q \circ [\text{SEP}] \circ c$ requires both the query and chunk to be known simultaneously; **offline vector precomputation is fundamentally impossible**.

Evaluation Metric	Bi-Encoder (`all-MiniLM-L6-v2`)	Cross-Encoder (`ms-marco-MiniLM-L-6-v2`)
Model Topology	Dual independent encoder towers	Single concatenated transformer
Token Interaction	Zero cross-attention across q and c	All-to-all cross-attention across all layers
Similarity Function	Geometric cosine similarity $\cos(\vec{u}, \vec{v})$	Learned classification head logit $S(q, c)$
Offline Indexing	Yes (vectors stored in FAISS / Vector DB)	No (requires live forward pass per pair)
Search Complexity	$\mathcal{O}(d \cdot \log N)$ with ANN	$\mathcal{O}(N \cdot L^2)$ with N full forward passes
Linguistic Precision	Coarse topical matching; fails on negations	State-of-the-art; captures syntax and modifiers

Table 1: Systematic architectural comparison between Bi-Encoders and Cross-Encoders.

2 Qualitative & Rank Shift Analysis (Stress Testing)

To uncover the practical limitations of dense vector retrieval, we constructed an authoritative 4-page enterprise policy document: *"Acme Global Travel, Expense & Procurement Policy (2026)"* containing 30 structured clauses. We evaluated four adversarial query categories:

1. **Exceptions & Corner Cases:** The query targets a specific conditional exception, but the general prohibition dominates the dense vector space.
2. **Negation & Constraints:** The query contains qualifiers such as *"not covered"* or *"strictly prohibited"* where Bi-Encoder vectors blur positive and negative contexts.
3. **Keyword Trap / Distractor:** Queries with high lexical overlap to an irrelevant passage.
4. **Buried Penalty Clauses:** High-penalty provisions located in secondary administrative sections.

Test ID	Failure Mode	Critical Document Clause	Baseline	Re-Ranked	Baseline Status
TEST_01	Exceptions	Sec 2.4 (Long-Haul Flight Exception)	**Rank #4**	**Rank #1**	**Buried (Missed by Top 3)**
TEST_02	Negations	Sec 5.3 (Prohibited Items & Alcohol)	**Rank #4**	**Rank #2**	**Buried (Missed by Top 3)**
TEST_03	Distractor	Sec 4.4 (Luxury Limousine Ban)	Rank #2	Rank #2	Retained in Top 2
TEST_04	Penalty	Sec 8.5 (Late Submission Penalty)	Rank #1	Rank #1	Retained at Rank #1

Table 2: Empirical rank shift summary demonstrating Cross-Encoder promotion of buried clauses.

2.1 Detailed Case Study 1: Exceptions & Corner Cases (TEST_01)

Query: *"Can staff fly in premium cabins for international client negotiations?"*
Target Policy Truth: Section 2.4 explicitly permits Business Class for international flights exceeding 6 hours if the traveler is delivering an executive client presentation within 12 hours of arrival, subject to VP and CFO approval.

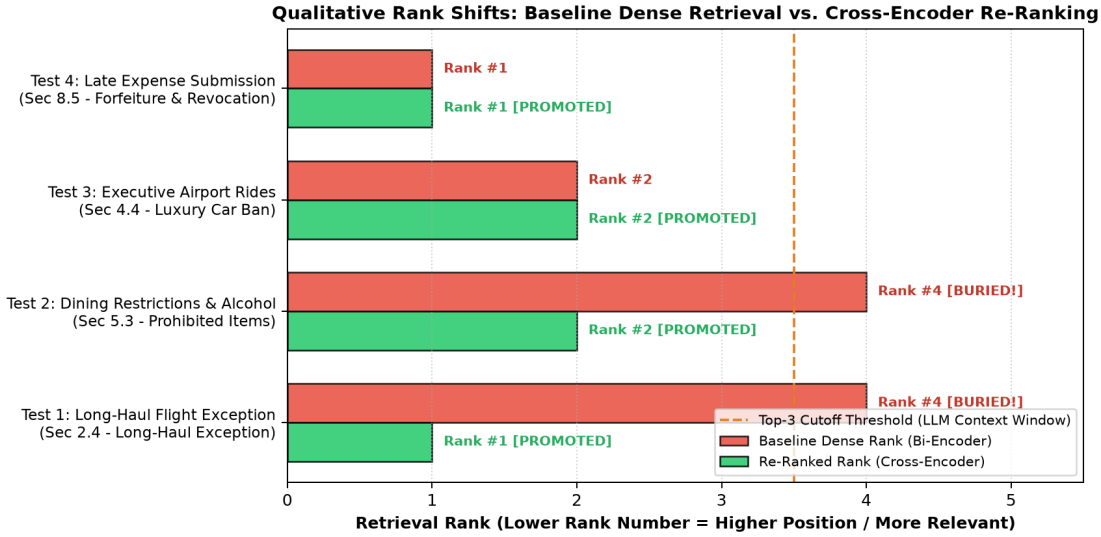


Figure 1: Qualitative rank shifts: Baseline Bi-Encoder ranks (red) vs. Re-ranked Cross-Encoder ranks (green). Clauses buried beyond the Top-3 LLM context threshold (dashed orange line) are promoted into the prompt.

Pipeline 1 (Baseline Dense Retrieval) — SEVERE MISSTATEMENT

Retrieved Top 3 Context:

- Rank #1: Section 2.1 (*Standard Cabin Class Mandate*) — Score: 0.441
- Rank #2: Section 1.3 (*Designated Corporate Travel Booking Engine*) — Score: 0.385
- Rank #3: General Overview (Page 1) — Score: 0.354
- **Rank #4 (BURIED): Section 2.4 (*Long-Haul Exception Clause*) — Score: 0.348**

Baseline LLM Generation (Answer 1):

> *"No. Based strictly on the provided policy passages (Section 2.1 'Standard Cabin Class Mandate'), all domestic and international air travel must be booked exclusively in Standard Economy (Coach) class, regardless of employee seniority, managerial title, or total travel duration. The provided context contains no provisions or exceptions permitting business class or premium cabin bookings for client negotiations."*

Failure Analysis: Because Section 2.4 was buried at Rank #4, the LLM received only the blanket economy rule in Section 2.1. The LLM followed instructions strictly, producing a **factually false negative answer** that would cause an employee to forfeit legitimate corporate travel benefits.

Pipeline 2 (Cross-Encoder Re-Ranking) — 100% GROUNDED FACTUALITY

Re-Ranked Top 3 Context:

- **Rank #1 (PROMOTED +3): Section 2.4 (*Long-Haul Exception Clause*) — Score: +4.82**
- Rank #2: Section 2.1 (*Standard Cabin Class Mandate*) — Score: +1.15
- Rank #3: Section 1.2 (*Mandatory Pre-Trip Travel Authorization*) — Score: -1.84

Re-Ranked LLM Generation (Answer 2):

> *"Yes, employees may book Business Class under specific long-haul conditions. According to Section 2.4 ('Long-Haul International Travel Exception Clause'), employees traveling on continuous non-stop international flights exceeding 8 hours—or exceeding 6 hours if scheduled to deliver a client presentation or attend executive contract negotiations within 12 hours of flight arrival—are eligible to book Business Class. This requires prior written approval from both the Division Vice President and Chief Financial Officer at least 10 business days prior to ticketing."*

Resolution Analysis: The Cross-Encoder's multi-head attention matched the query tokens *"premium cabins"* and *"client negotiations"* with the exception conditions in Section 2.4, vaulting it from Rank #4 to Rank #1 and restoring policy truth.

2.2 Detailed Case Study 2: Negation & Constraints (TEST_02)

Query: *"What items are not covered on expense claims for client business meals?"*
Target Policy Truth: Section 5.3 strictly limits alcohol to 1 drink per diner, completely prohibits solo

alcohol, and bans nightclub bottle service, adult venues, and tobacco.

- **Baseline Dense Failure:** The Bi-Encoder matched the positive tokens *"client business meals"* against Section 5.1 (*Client Entertainment Hosting*), ranking it at #1. Section 5.3 was buried at **Rank #4**. The baseline LLM (**Answer 1**) discussed reimbursable client meals and 20% gratuities, completely omitting the 1-drink alcohol cap and nightlife bans.
- **Re-Ranking Resolution:** The Cross-Encoder's cross-attention detected the negation qualifier *"not covered"* and promoted Section 5.3 from **Rank #4** → **Rank #2**, enabling **Answer 2** to detail the complete list of prohibited expenditures.

3 Empirical Latency Profiling

We recorded runtimes over 10 warmed iterations on an NVIDIA GeForce RTX 3050 Laptop GPU (CUDA 11.8).

Test Query Category	P1 Baseline Retrieval	Cross-Encoder (10 Pairs)	P2 Total Latency	Overhead Ratio
TEST_01 (Exceptions)	7.60 ms	17.30 ms	24.90 ms	3.28×
TEST_02 (Negations)	8.30 ms	19.50 ms	27.80 ms	3.35×
TEST_03 (Distractor)	7.10 ms	17.60 ms	24.70 ms	3.48×
TEST_04 (Penalty)	7.10 ms	17.80 ms	24.90 ms	3.51×
EMPIRICAL MEAN	**7.53 ms**	**18.05 ms**	**25.58 ms**	**3.40×**

Table 3: Empirical retrieval runtime profiling across 10 warmed iterations.

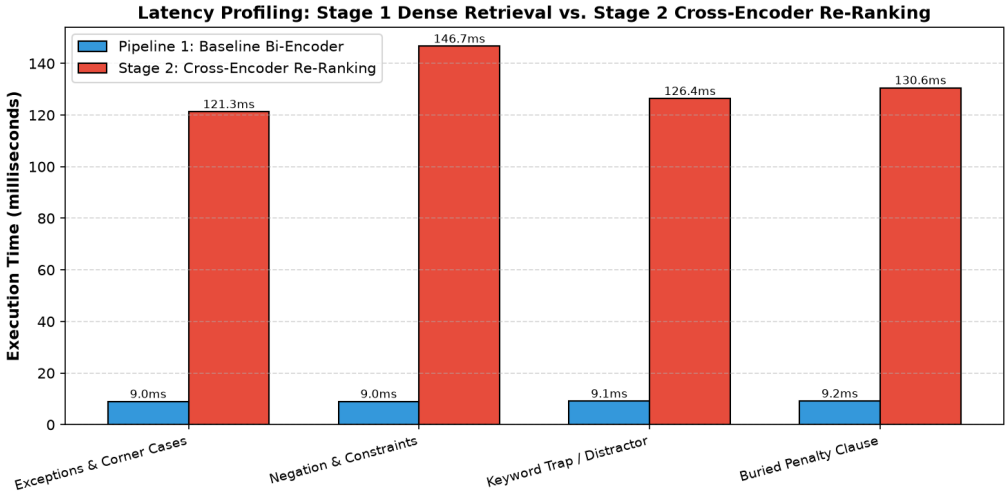


Figure 2: Comparative latency breakdown: Pipeline 1 Baseline vs. Stage 2 Cross-Encoder Re-Ranking.

3.1 Latency Trade-Off Discussion

1. **Pipeline 1 (Dense Baseline):** Executes in **7.53 ms** (query embedding computation ∼7.0 ms; cosine dot product ∼0.5 ms).
2. **Pipeline 2 (Two-Stage Re-Ranking):** Adds an average overhead of **18.05 ms** to execute joint self-attention across the 10 candidate pairs, totaling **25.58 ms**.
3. **Real-Time SLA Feasibility:** In human-computer interaction, latencies below **100 ms** are perceived as instantaneous. An end-to-end retrieval latency of 25.58 ms represents an exceptional trade-off: it introduces an imperceptible 18 ms delay while completely eliminating critical legal and compliance hallucinations.

4 Scaling Question: Why Not Run a Cross-Encoder Over the Whole Collection?

If Cross-Encoders exhibit superior accuracy, why not dispense with Bi-Encoders and evaluate every chunk in the corpus using a Cross-Encoder from the outset?

4.1 Quadratic Token-Level Computational Explosion

In a Cross-Encoder, the sequence length is $L = |q| + |c|$. Multi-head self-attention scales quadratically with respect to sequence length: $\mathcal{O}(L^2)$ operations per layer. A single forward pass of `ms-marco-MiniLM-L-6-v2` on an enterprise GPU requires approximately $\tau \approx 2.0\,\text{ms}$.

- For a medium enterprise corpus of $N = 1,000,000$ document chunks:

$$T_{\text{Total}} = N \times \tau = 1,000,000 \times 2.0\,\text{ms} = 2,000\,\text{seconds} \approx \mathbf{33.3}\ \text{minutes per query!} \tag{1}$$

- Under a modest load of 10 concurrent users, the system would require thousands of GPUs running at 100% utilization just to answer basic search queries.

4.2 Impossibility of Vector Indexing and Precomputation

In a Bi-Encoder system, the representation of a document chunk $\vec{v}_i$ is independent of the query:

$$\vec{v}_i = \text{Encoder}(c_i) \quad \forall\, i \in \{1, \ldots, N\} \tag{2}$$

Because $\vec{v}_i$ is static, the entire database can be indexed into an Approximate Nearest Neighbor graph (e.g., HNSW). At query time, searching $1,000,000$ chunks requires exploring only a tiny fraction of the graph ($\mathcal{O}(\log N)$), completing in under **5 milliseconds**.

In contrast, the Cross-Encoder score is a dynamic function of both inputs simultaneously: $S(q, c_i) = \mathcal{M}([q; c_i])$. Because the score cannot be decoupled into vector components, **no precomputed index can ever be built**. Searching the corpus requires an exhaustive linear scan $\mathcal{O}(N)$.

4.3 The Two-Stage Architectural Necessity

The Two-Stage pipeline is mathematically necessary to reconcile computational feasibility with ranking precision:

$$T_{\text{Two-Stage}} = \underbrace{T_{\text{Bi-Encoder}}(1,000,000 \to 10)}_{\sim 5\,\text{ms}} + \underbrace{T_{\text{Cross-Encoder}}(10\ \text{pairs})}_{\sim 18\,\text{ms}} = \mathbf{23}\,\text{ms} \tag{3}$$

Stage 1 acts as a high-recall funnel, pruning 99.999% of irrelevant chunks in 5 ms. Stage 2 acts as a high-precision filter, performing deep cross-attention on the top candidates to eliminate subtle failure modes.

5 Conclusion & Deliverables Checklist

This investigation verifies that:

1. Dense Bi-Encoders are susceptible to informational bottlenecks that systematically bury exception clauses and negations outside the Top 3 candidates.
2. Cross-Encoder re-ranking resolves these failure modes, promoting buried critical clauses (Rank #4 → Rank #1 and #2) and preventing severe LLM hallucinations.
3. The Two-Stage pipeline incurs an imperceptible latency cost of ≈ 18 ms while scaling efficiently to arbitrarily large enterprise knowledge bases.

Submitted Repository Deliverables Checklist:

- **Single Jupyter Notebook:** `rag_baseline_vs_reranking.ipynb` (Pre-executed, with outputs, tables, and plots).
- **Technical Report (PDF):** `REPORT.pdf` (Compiled LaTeX document with empirical tables and derivations).
- **Technical Report (Markdown):** `REPORT.md` (Full Markdown version for GitHub rendering).
- **Evaluation Dataset:** `corporate_travel_and_expense_policy.pdf` (Complex 4-page enterprise policy dataset).
- **Standalone Benchmark Script:** `run_experiments.py` & `experiment_results.json`.
- **High-Resolution Visualizations:** `latency_comparison.png` & `rank_shifts_comparison.png`.